

Exercise 2: E-commerce Platform Search Function

Big O Notation is used to measure the performance of an algorithm as the input size increases.

- Best Case: Element found immediately $\rightarrow O(1)$
- Average Case: Element found in the middle $\rightarrow O(n)$
- Worst Case: Element found at the end or not present $\rightarrow O(n)$

Product.java

```
public class Product {
    int productId;
    String productName;
    String category;

    public Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    @Override
    public String toString() {
        return "ID: " + productId +
            ", Name: " + productName +
            ", Category: " + category;
    }
}
```

SearchDemo.java

```
import java.util.Arrays;
import java.util.Comparator;

public class SearchDemo {

    // Linear Search

    public static Product linearSearch(Product[] products, String name) {
        for (Product product : products) {
            if (product.productName.equalsIgnoreCase(name)) {
                return product;
            }
        }

        return null;
    }
}
```

```
}
```

```
// Binary Search
```

```
public static Product binarySearch(Product[] products, String name) {
```

```
    int left = 0;
```

```
    int right = products.length - 1;
```

```
    while (left <= right) {
```

```
        int mid = (left + right) / 2;
```

```
        int compare =
```

```
            products[mid].productName.compareToIgnoreCase(name);
```

```
        if (compare == 0) {
```

```
            return products[mid];
```

```
        } else if (compare < 0) {
```

```
            left = mid + 1;
```

```
        } else {
```

```
            right = mid - 1;
```

```
        }
```

```
    return null;
```

```
}
```

```
public static void main(String[] args) {
```

```
    Product[] products = {
```

```
        new Product(101, "Laptop", "Electronics"),
```

```
        new Product(102, "Phone", "Electronics"),
```

```
        new Product(103, "Shoes", "Fashion"),
```

```
        new Product(104, "Watch", "Accessories"),
```

```
        new Product(105, "Tablet", "Electronics")
```

```

};

String searchItem = "Watch";

System.out.println("=== Linear Search ===");

Product result1 = linearSearch(products, searchItem);

if (result1 != null)

    System.out.println(result1);

else

    System.out.println("Product Not Found");

Arrays.sort(products,

    Comparator.comparing(p -> p.productName));

System.out.println("\n=== Binary Search ===");

Product result2 = binarySearch(products, searchItem);

if (result2 != null)

    System.out.println(result2);

else

    System.out.println("Product Not Found");}}

```

Output:

=== Linear Search ===

ID: 104, Name: Watch, Category: Accessories

=== Binary Search ===

ID: 104, Name: Watch, Category: Accessories

Analysis:

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

- Linear Search checks products one by one, whereas Binary Search divides the search space into halves. Therefore, Binary Search is more efficient for large product datasets.

Exercise 7: Financial Forecasting

Recursion:

Recursion is a technique where a method calls itself to solve smaller instances of the same problem.

FinancialForecast.java

```
public class FinancialForecast {  
    // Recursive method  
    public static double futureValue(double currentValue,  
        double growthRate,  
        int years) {  
        if (years == 0) {  
            return currentValue;  
        }  
        return futureValue(currentValue,  
            growthRate,  
            years - 1) * (1 + growthRate);  
    }  
    public static void main(String[] args) {  
  
        double currentValue = 10000;  
        double growthRate = 0.10; // 10%  
        int years = 5;  
  
        double predictedValue =  
            futureValue(currentValue, growthRate, years);  
        System.out.printf(  
            "Predicted Future Value after %d years = %.2f",  
            years,  
            predictedValue);  
    }  
}
```

Output:

Predicted Future Value after 5 years = 16105.10

Analysis:

Time Complexity:

Recursive Calls = n

Time Complexity = $O(n)$

Optimization:

The recursive solution can be optimized using:

- Memoization
- Dynamic Programming

These techniques reduce repeated computations and improve performance.

Why Recursion?

Recursion makes the forecasting logic simple and easy to understand, especially when calculations depend on previous years' values.